Day 2 Deliverable: Dataset Sources, Input Requirements and Validation Mechanism

Insider Threat Detection System, Capstone Group 7 · 5 November 2026

1. Purpose

This document is the Day 2 deliverable. It answers both columns of the task table.

Column	Day 2 task	Where answered
Research and methodology	Document dataset sources, collection procedures or input-data requirements	Sections 2, 3 and 4
Prototype development	Implement data import, preprocessing and validation mechanisms	Sections 5 and 6

Evidence (code, tests, a clean run and a rejected run) is in Section 7. Limits are in Section 8.

2. Dataset sources

Three sources feed the same pipeline. All of them are read in the CERT folder layout (Section 4).

Source	What it is	Role in this project	Status
Local generator (scripts/generate_synthetic_cert.py)	Generated records in the CERT layout. Defaults: 120 users, 90 days, 12 insiders, seed 7. The automated tests use 60 users, 80 days and 8 insiders.	Functional checks, tests and the demonstration	In use. The run in this deliverable has 108,635 events, 12 insiders and 681 malicious events.
CERT r4.2	Synthetic background activity and malicious-actor scenarios published by the CERT Insider Threat Center at Carnegie Mellon University (reference D1 in the Word report). The code is written for its size of about 32.8 million events.	Later benchmark stage	Not yet run

Source	What it is	Role in this project	Status
SPEDIA	Exercise attacks by participants, simulated benign activity and sampled CERT events, in one CSV file (reference D2 in the Word report)	Later exploratory cross-dataset test	Not yet run. An adapter exists (itd/spedia.py).

Two cautions apply to every result in this project. Generated data can contain patterns that are easy to learn, so it shows that the pipeline works but not that it generalizes. The share of insider users must not be confused with the share of malicious events or malicious sessions.

3. Collection procedures

Source	Steps	Command or note
Local generator	1. Generate the five event files and answers.csv into one folder. 2. Validate. 3. Run the engine.	python scripts\generate_synthetic_cert.py data\cert_synthetic, then python -m itd.validate data\cert_synthetic
CERT r4.2	1. Obtain the release from its publisher and read its terms of use. 2. Copy logon, device, email, file and http CSV files into one folder. 3. Build answers.csv from the release's own answer files only. 4. Record the release name, the SHA-256 of each file, row counts and the time range. 5. Validate. 6. Run the engine.	python scripts\build_answers.py <answers-folder> <dataset-folder>. Hashes: Get-FileHash -Algorithm SHA256 <file>.

Source	Steps	Command or note
SPEDIA	1. Inspect the single CSV to see its columns and event types. 2. Copy spedia_mapping.example.json and set the real column and event-type names. 3. Convert to the CERT layout. 4. Validate. 5. Run the engine.	python -m itd.spedia --inspect logs_SPEDIA.csv, then python -m itd.spedia --csv logs_SPEDIA.csv --map spedia_mapping.json --out data\spedia_cert
Privacy (any source)	User ids can be replaced by keyed hashes at ingestion.	python -m itd.engine --data <folder> --pseudonymize (key in PSEUDONYM_KEY)

The answer builder must receive only the answer files of the release in use. Missing labels must not be read as confirmed benign data.

4. Input-data requirements

File	Required columns	Allowed values and format
logon.csv	id, date, user, pc, activity	activity is Logon or Logoff
device.csv	id, date, user, pc, activity	activity is Connect or Disconnect
email.csv	id, date, user, pc, to, size, attachments	size and attachments are numbers (size in bytes in the source, reported in KB)
file.csv	id, date, user, pc, filename	none
http.csv	id, date, user, pc, url	none
answers.csv (optional)	activity_type, id	ids of malicious events

General requirements:

- All five event files are present in one folder. Extra columns are ignored.
- Dates use MM/DD/YYYY HH:MM:SS. No time zone is handled; all times are read as given.
- Event ids are unique across all files, because labeling matches ids alone.
- id, user and pc are never empty.

- For training, the time-ordered parts (70% training, 15% validation, 15% test) each need at least one malicious session. Without answers.csv the engine can only score with an already saved model.
- Files are read in chunks of 500,000 rows. The logon data and the unique-value counters stay in memory, so memory use on a large release must be measured, not assumed.

5. Prototype: the validation mechanism

5.1 How it works

A new module, itd/validate.py, checks the data folder before the engine reads any of it. It is step 0 of the engine and can also be run alone.

1. The engine calls the validator first and prints the report.
2. If any error is found, the run stops with the message "Input validation failed; fix the errors above and run again." Nothing is read, trained or written to the database.
3. Warnings are printed and the run continues. The data is accepted exactly as the ingestion code reads it.
4. The full report is saved as validation.json in the report folder, and a short summary (status, error count, warning count, rows per file) is stored in the run document under the field validation.

Errors are the conditions that previously ended the run with a raw pandas or Python error. Warnings are the conditions that previously passed silently. The validator now names them, counts them and gives example line numbers (line 1 is the header), without changing what the engine does with the data.

5.2 Rules

Rule	Severity	Check	Before Day 2
V1	Error	A required CSV file is missing	Stopped with a missing-file message (one file list)
V2	Error	A required column is missing	Raw pandas error
V3	Error	A date does not match MM/DD/YYYY HH:MM:SS	Raw pandas error without the line
V4	Warning	id, user or pc is empty	Silent

Rule	Severity	Check	Before Day 2
V5	Warning	An event id appears more than once, inside a file or across files	Silent
V6	Warning	Unknown activity value in logon.csv or device.csv (the event is ignored)	Silent
V7	Warning	Email size or attachments is not a number (counted as 0)	Silent
V8	Error	answers.csv has no id column	Raw error
V9	Warning	Answer ids that match no event (those labels are lost)	Silent
V10	Warning	answers.csv is missing or empty, so every session is labelled 0 and training will stop	Stopped later at training

V10 is a warning, not an error, because a saved model can still score data that has no labels.

5.3 Cost

The validator reads every file once more, so a run takes longer. The duplicate check keeps one 8-byte hash per event id; for 32.8 million events that is roughly 0.3 GB, plus a temporary copy while counting. This is an estimate and has not been measured on a large release.

6. Code changes on Day 2

File	Change
backend/itd/validate.py	New. Ten rules, report builder, text report and a command line (python -m itd.validate <folder> [report.json]).
backend/tests/test_validation.py	New. Nine tests: clean data, every rule, the report text and the engine stopping before it reads data.
backend/itd/engine.py	Four edits: import, validation step before the database is opened, a validation summary in the run document, and validation.json in the report folder.

Test results: 42 passed and 1 skipped before Day 2; 51 passed and 1 skipped after, which is the 9 new tests. The skipped test needs a real MongoDB.

7. Evidence

Figure 1 shows the validator being created in the repository and compiling (validate.py OK).

```
les>>
>>
>>
>> def format_report(rep):
>>     lines = [f"    Input validation: {rep['status'].upper()}  " + ", ".join(f"{k}.csv {v:,}" for k, v in rep[
"].items())]
>>     for i in rep["issues"]:
>>         where = f" (lines {', '.join(map(str, i['example_lines']))})" if i["example_lines"] else ""
>>         lines.append(f"    [{i['rule']} {i['severity']}] {i['file']}: {i['message']} - {i['count']:,}{where}"
>>     return "\n".join(lines)
>>
>>
>> if __name__ == "__main__":
>>     if len(sys.argv) < 2:
>>         raise SystemExit("usage: python -m itd.validate <data folder> [report.json]")
>>     report = validate_folder(sys.argv[1])
>>     print(format_report(report))
>>     if len(sys.argv) > 2:
>>         with open(sys.argv[2], "w") as f:
>>             json.dump(report, f, indent=2)
>>     raise SystemExit(1 if report["status"] == "failed" else 0)
>> '@ | Set-Content -Encoding UTF8 itd\validate.py
(.venv) PS C:\Users\f\Desktop\itd_clean\backend> python -m py_compile itd\validate.py; "validate.py OK"
validate.py OK
(.venv) PS C:\Users\f\Desktop\itd_clean\backend> @'
>> """Pre-flight input validation (itd/validate.py): every rule V1 to V10, and the engine stops on errors."""
>> import os
>> import subprocess
>> import sys
>>
>> os.environ.setdefault("MONGO_URI", "mock")
>> import pandas as pd
>> import pytest
>>
>> ROOT = os.path.dirname(os.path.dirname(__file__))
>>
>>
>> @pytest.fixture(scope="module")
>> def clean(tmp_path_factory):
>>     d = tmp_path_factory.mktemp("clean")
```

Figure 1. Creation of the validator.

Figure 2 shows the test file being created and compiling (tests OK).

```
>>
>>
>> def test_engine_stops_before_reading_when_validation_fails(data):
>>     from itd import engine
>>     os.remove(data / "device.csv")
>>     with pytest.raises(SystemExit, match="validation failed"):
>>         engine.run(str(data), retrain=True, report_dir=str(data / "rep"))
>> '@ | Set-Content -Encoding UTF8 tests\test_validation.py
(.venv) PS C:\Users\f\Desktop\itd_clean\backend> python -m py_compile tests\test_validation.py; "tests OK"
tests OK
(.venv) PS C:\Users\f\Desktop\itd_clean\backend>
```

Figure 2. Creation of the validation tests.

Figure 3 shows the engine patch and the four matching lines in engine.py: the import, the validate_folder call, the report line and validation.json.

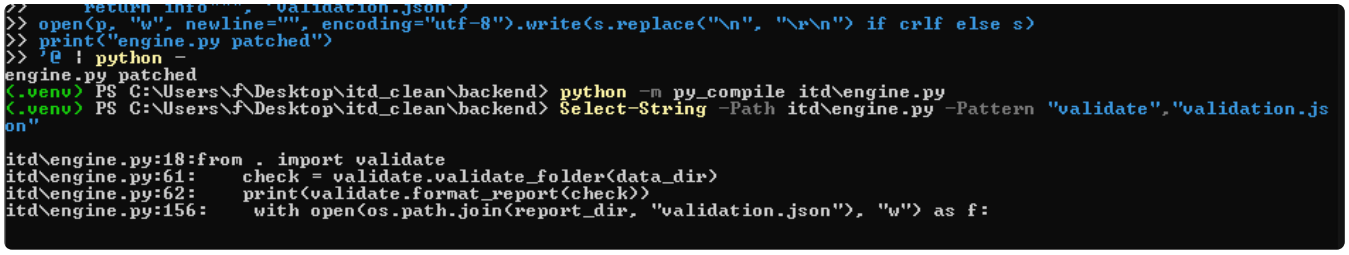


Figure 3. The engine patch.

Figure 4 shows the full test run (51 passed, 1 skipped), the generated data (108,635 events, 12 insiders, 681 malicious events) and the validator accepting it: Input validation: OK with the row count of every file.

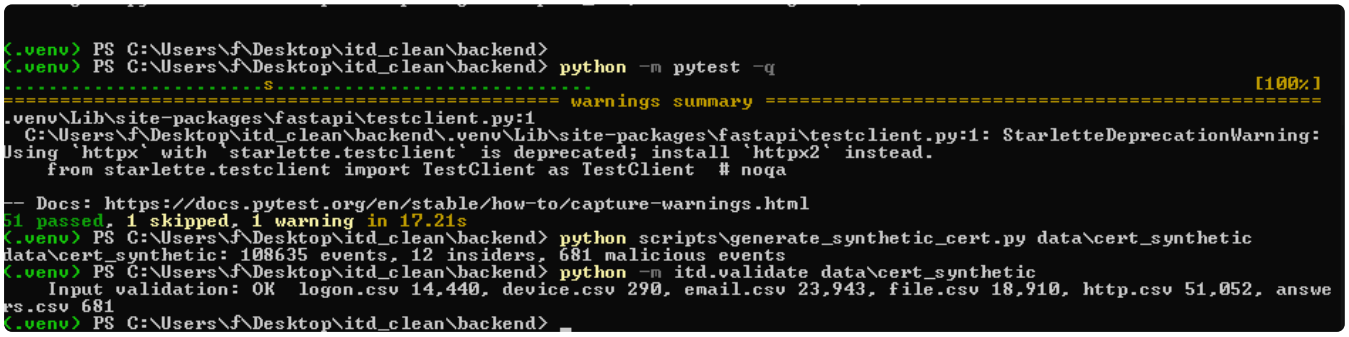


Figure 4. Test run, generated data and a passing validation.

Rejected input. The file device.csv was removed from a copy of the data. The validator and the engine both refused it:

```
> python -m itd.validate data\cert_broken
    Input validation: FAILED
    [V1 error] device.csv: a required CSV file is missing: device.csv - 1

> python -m itd.engine --data data\cert_broken
0/5 Input validation ...
    Input validation: FAILED
    [V1 error] device.csv: a required CSV file is missing: device.csv - 1
Input validation failed; fix the errors above and run again.
```

The other rules (V2 to V10) are covered by the automated tests, each with damaged data: a removed column, a changed date on a known line, an empty user, an unknown activity, a non-numeric size, duplicate ids inside and across files, and three damaged answers files.

8. Limits and open items

- Closed on 6 November 2026 against a real MongoDB: the run document stored the validation summary (status ok, 0 errors, 0 warnings, row counts of all files), and a run on a folder without device.csv was rejected by rule V1 (see the Day 3 document, Figures 5 to 8).

- The test that needs a real MongoDB is skipped by default.
- Original CERT r4.2 and SPEDIA have not been run. The validator has not been timed or measured for memory on files of that size.
- The report in Section 14 also asks for file hashes, the time range, time-zone assumptions, orphan logoffs and boundary ties. The validator does not record these yet; file hashes are a manual step in Section 3.
- Validation checks structure, not meaning. It cannot tell whether the labels are correct.

9. Next step (Day 3)

Column	Day 3 task
Research and methodology	Describe data-processing steps, algorithm configuration and output generation
Prototype development	Integrate the main processing module with the system interface (run endpoint, run status and a run button in the dashboard)